FYTA — Data Engineer assessment

Sagar Marthandan

Three weeks from 8 soil sensors — about 16,000 readings — plus 49 user notes and 28 plant photos. The job: find what is wrong with the data, what is real, and what a model could safely use. `uv run fyta_pipeline.py` reproduces everything.

Five things that matter

1. A sensor stopped measuring, and nothing says so. Sensor 6's moisture reads exactly 25.01 for fourteen days straight — 1,344 readings — and looks perfectly normal. What gives it away is that the *same* device's four other measurements kept moving. A real pot does not hold still while everything else changes.

2. A two-day gap that leaves no trace. One sensor is missing 193 readings. There are no blank cells and no errors — the rows simply are not there. Counting empty cells finds nothing; you see it only by working out how many readings *should* exist and subtracting.

3. A sensor in the wrong units, on the wrong clock. Sensor 7's temperature was Fahrenheit, not Celsius, and its clock was two hours out. Neither is visible in the values. Fahrenheit shows up only because its readings jitter 1.8× faster than the others; the clock only by comparing its daylight pattern against the other seven.

4. One join is safe, the next is not. Plant IDs repeat: one ID covers more than one sensor. Joining the sensor table is fine; joining the notes or photos to it silently multiplies rows — 49 become 67. The trap: the first join works, so the next looks safe too.

5. Two pots have two sensors each, and the pairs disagree. Probes in one pot share the same soil, so their readings should rise and fall together. They do not. Either they are not really in one pot, or the mapping is wrong. **I could not settle this from the data alone.**

Three decisions I would defend

Not everything out of range is wrong. A physically impossible reading is removed and logged; a merely unusual one is *kept* and marked. Treating those the same is how real events get deleted — one sensor has 431 unusual readings, all in long stretches.

Finding a problem and fixing it are separate jobs, and I check both. A detector with no fix is a warning nobody acts on; a fix with no detector rots. The gate before publishing also proves every removed value really was removed.

Never average the two sensors on one pot. An average describes neither pot, and throws away the disagreement — which is the only reason to fit two probes.

What I would never ship

- **Averaged sensors on one pot** — the disagreement *is* the signal.
- **A filled-in gap** — a user note sits inside those missing two days. Inventing numbers there would fabricate readings that were never taken.
- **Trimmed unusual readings** — 107 % moisture is a finding; clipping it to the normal ceiling would be a lie.
- **Deleted odd readings** — eight light readings at 1am look wrong by the clock but sit inside range. Someone may have switched a lamp on. Kept, marked.
- **A safety check that cannot fail** — see below.

Where I was wrong

A second review found four real defects *after* I had reported them working. All are fixed, each with a test that fails if the fix is undone:

- **The publishing gate could not fail.** It asked "is any value missing?", which any missing value satisfies. I restored 1,344 known-bad readings; it passed them.
- **A counting error treated the start of each sustained excursion as an isolated sample.** The corrected result is 11 runs holding all 431 out-of-spec readings, with zero isolated. The cause of that sensor's behaviour is still unresolved.
- **A falling reading counted as agreement.** The test asked whether fertiliser raised conductivity; a large *drop* scored as support. It now requires a rise.
- **A photo taken before an event counted as a response to it.** The window was symmetric; it now looks only forward, and "no photo after the event" is its own answer rather than a silent zero.

Several findings also carried numbers no code produced; those now come from detectors, and two changed. The pattern each time: I checked by re-reading my own writing, not by re-running.

What I would do next

1. **Put the checks in the build** — every new check gets a test that fails without it.
2. **Move the device settings into an effective-dated registry.** Unit fixes, clock offsets and sensor specifications are code today; plant, soil type and species are already data. Making the rest data means a new sensor family is a row, not a release.
3. **Settle the two-sensor question** — it decides how the modelling table is built.
4. **Version each derived value.** The moisture figure comes from a soil-type curve, and that curve has no version and no date.